

Class 10

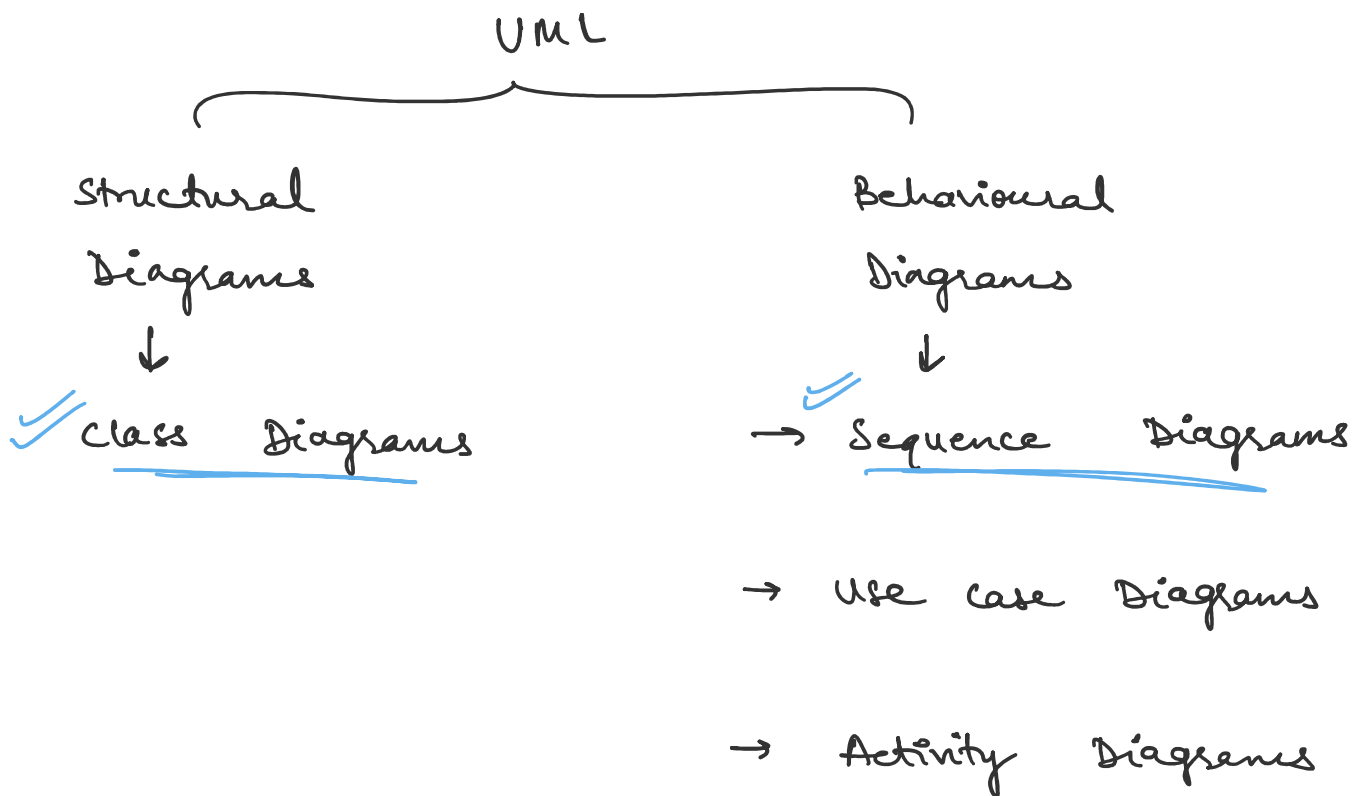
Friday, June 26, 2026

9:07 PM

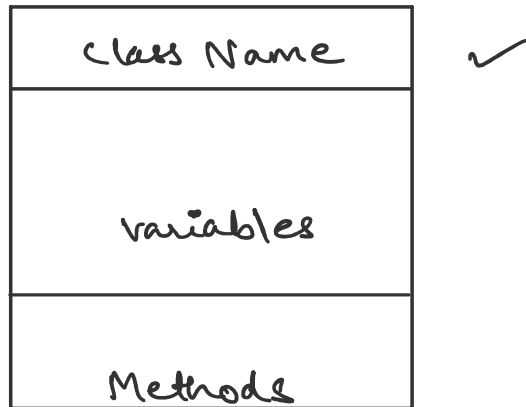
UML



Unified Modelling Language



Class Diagram



variables —

+ → public

- → private

→ protected

~ → Package default

```
class Student
{
    private int id;
```

private String name ;

private int age ;

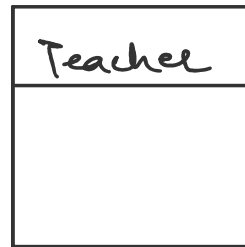
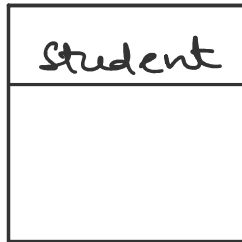
public void setName() ;

protected int getId() ;

private boolean validAge() ; → Above 18
check

}

Student	
- id : int	
- name : String	
- age : int	
+ setName () : void	
# getId () : int	
- validAge () : boolean	



Relationship

1. Association



customer places order

```
public class Customer
{
    order order;
    ==
}
```



2. Aggregation



has-A relationship



weak ownership

```
class Library
```

```
{
```

```
    List < Book > books ;
```

```
    ==
```

```
}
```



3. Composition



Strong Has - A relationship

```
class House
{
    Room room;
    ==
}
```



4. Inheritance



is - A relationship

Dog is an Animal.

```
class Animal
```

{

}

class Dog extends Animal

{

}



6. Interface

interface Payment

{

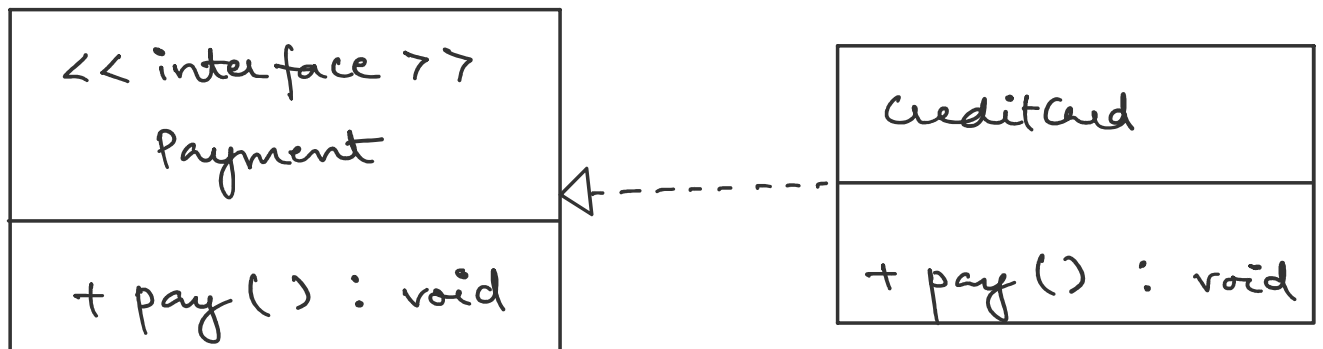
public void pay();

}

```

class CreditCard implements Payment
{
    ==
}

```



multiplicity → How objects are getting multiplied

→ Exactly one 1

→ 1 or many | ————— *

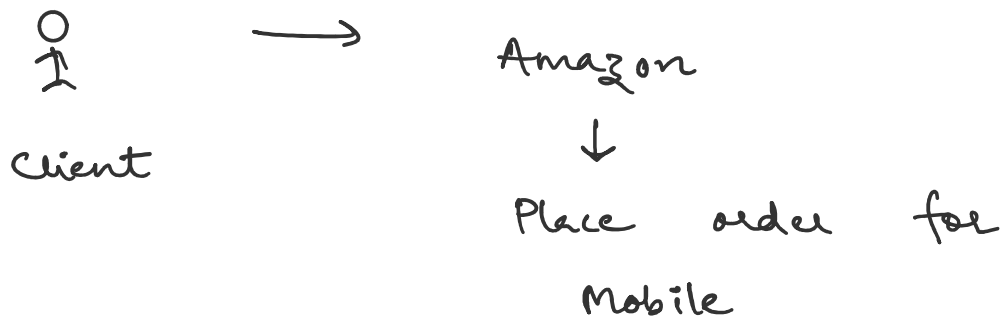


→ Zero or many ○ - - - - - *

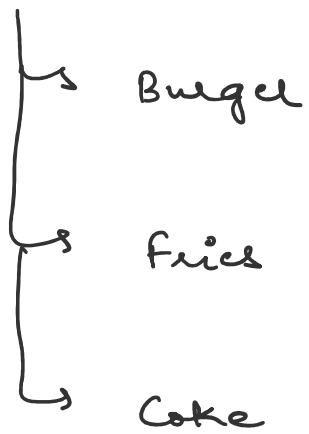
→ Zero or one ○ - - - - - 1

Abstract factory pattern

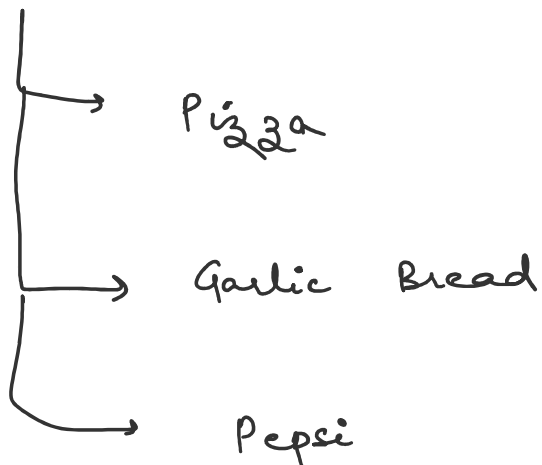
family of related objects → interface

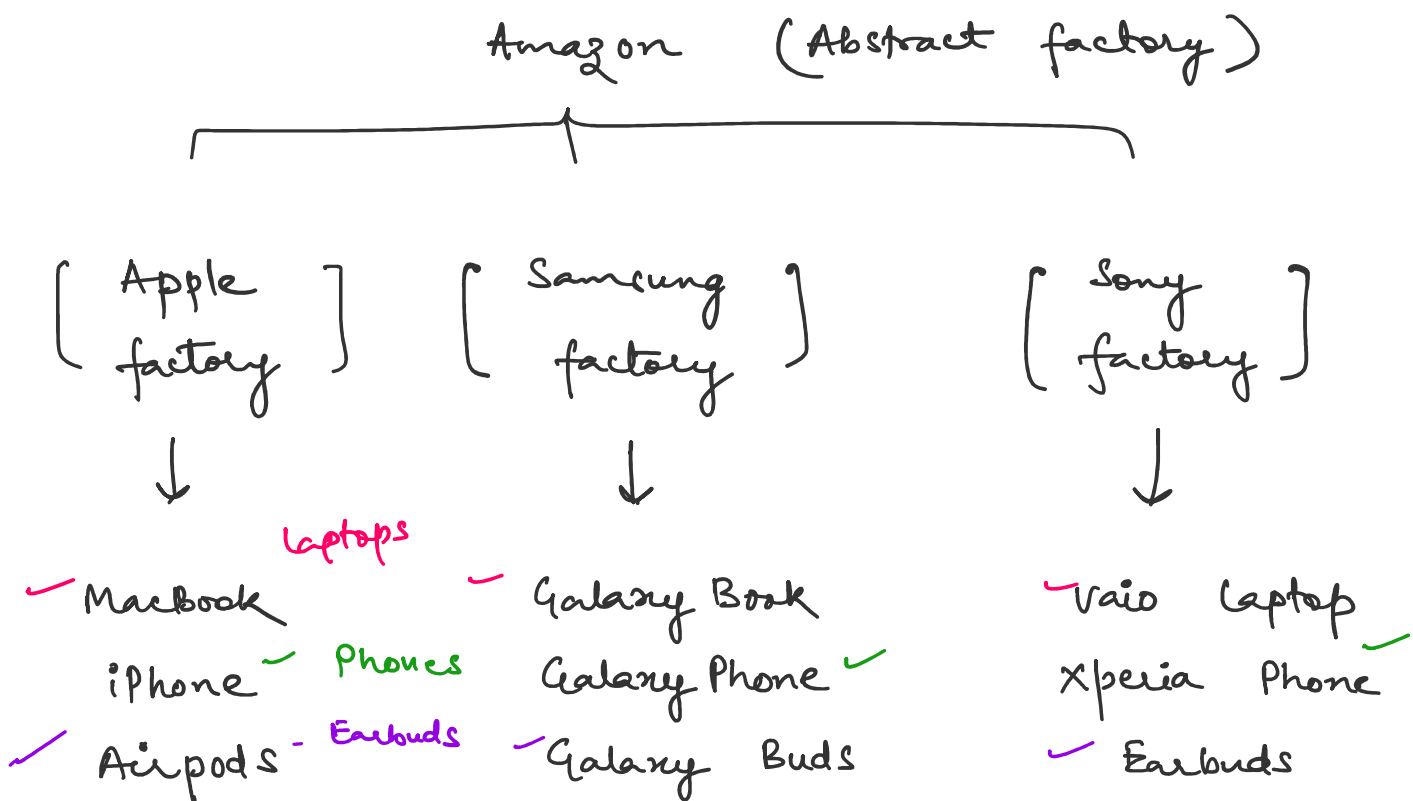
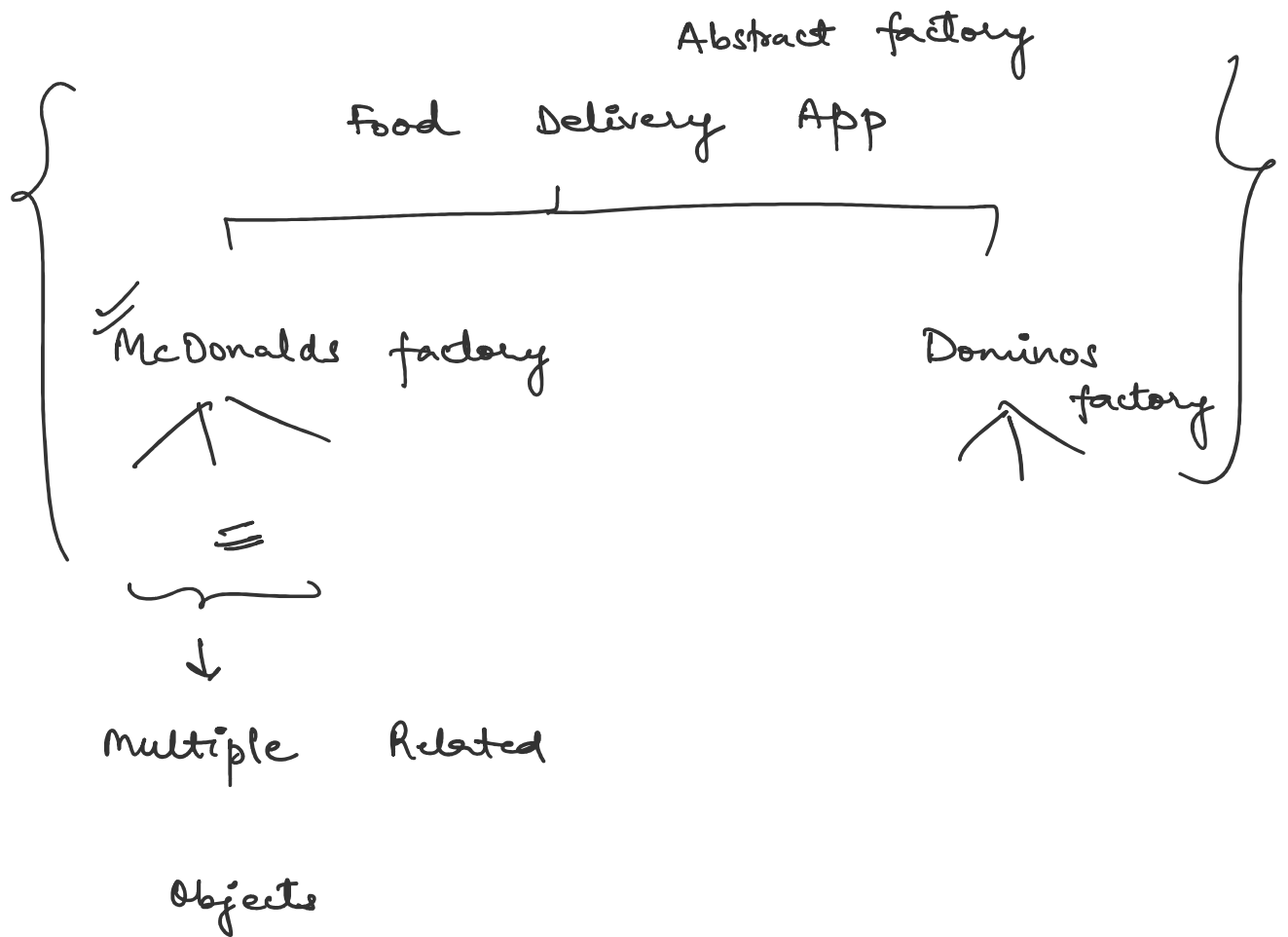


McDonalds //



Dominos //





steps -

1. create product interfaces

```
interface Laptop
{
    =
}
```

```
interface Phone    =
```

```
interface Earbuds  =
```

2. Create apple products

```
class MacBook implement Laptop
{
    =
}
```

3. create samsung products

4. create sony products

5. Define abstract factory

↓

interface BrandFactory

6. create apple factory

7. create samsung factory.

8. create sony factory

9. Client code → Main()

Notes:

What is UML?

UML (Unified Modeling Language) is a standard way to visually represent the design of a software system. It helps developers, architects, and stakeholders understand the structure and behaviour of an application before writing code.

UML is widely used in:

- Low-Level Design (LLD)
- High-Level Design (HLD)
- System documentation
- Software architecture discussions
- Technical interviews

Types of UML Diagrams

UML diagrams are mainly divided into two categories:

1. Structural Diagrams

These describe the static structure of the system.

Examples:

- Class Diagram (Most important for LLD)

- Object Diagram
- Package Diagram
- Component Diagram
- Deployment Diagram

2. Behavioural Diagrams

These describe how objects interact and behave.

Examples:

- Sequence Diagram (Most important for LLD)
- Use Case Diagram
- Activity Diagram
- State Machine Diagram
- Communication Diagram

For interviews, the two most important UML diagrams are:

- Class Diagram
- Sequence Diagram

Class Diagram

A Class Diagram represents the static structure of a system.

It shows:

- Classes
- Attributes
- Methods

- Relationships between classes

Think of it as the blueprint of your code.

Basic Structure of a Class

A class is represented using three sections.

Student
- id : int - name : String - age : int
+ study() + attendClass()

Top section

- Class Name

Middle section

- Attributes (Variables)

Bottom section

- Methods (Functions)

Visibility Symbols

Symbol	Meaning	Accessible From
+	Public	Everywhere
-	Private	Inside same class only
#	Protected	Same class + Child classes
~	Package Default	Same package

Example :

```
- name : String  
+ getName() : String  
# calculateSalary()  
~ printDetails()
```

Attributes

Attributes represent data members.

Example -

```
- id : int  
- name : String  
- salary : double
```

General syntax

visibility name : datatype

Example

```
- email : String  
+ balance : double
```

Methods

Methods represent behavior.

Example-

```
+ deposit(amount : double)
+ withdraw(amount : double)
+ getBalance() : double
```

General syntax

visibility method(parameters) : returnType

Relationship Types

Relationships show how classes are connected. There are six major relationships :

1. Association

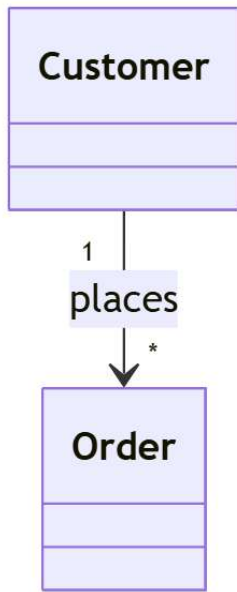
One object knows or uses another object. It is the most common relationship.

Example

A Customer places Orders. A customer can place many orders.

```
class Customer {
    Order order;
```

}



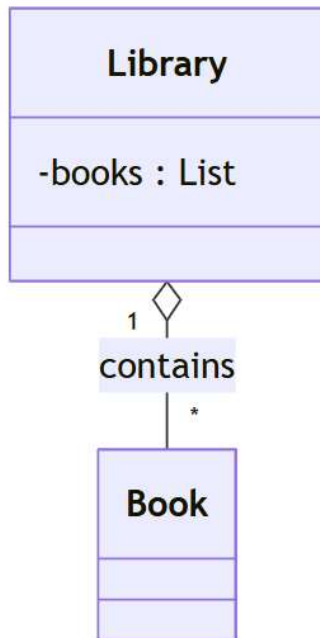
2. Aggregation

Has-a relationship with weak ownership. The child object can exist even if the parent is destroyed.

Example

Books can exist without a Library. If Library is deleted, Books still exist.

```
class Library {
    List<Book> books;
}
```



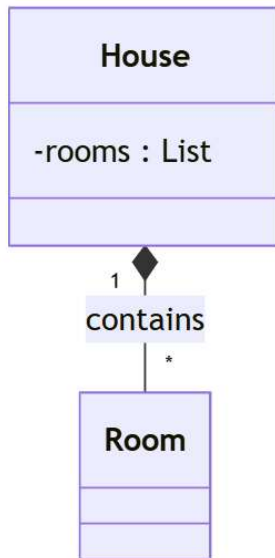
3. Composition

Strong Has-a relationship. Child cannot exist without Parent.

Example

A **House** has **Rooms**. If the **House** is deleted, all its **Rooms** are also deleted.

```
class House {
    List<Room> rooms = new ArrayList<>();
}
```



4. Inheritance (Generalization)

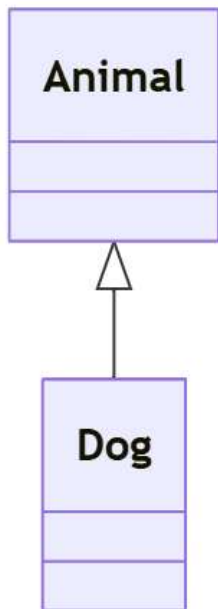
Represents an **is-a** relationship. A child class inherits the properties and behaviour of the parent class.

Example

Dog is an Animal.

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```



5. Dependency

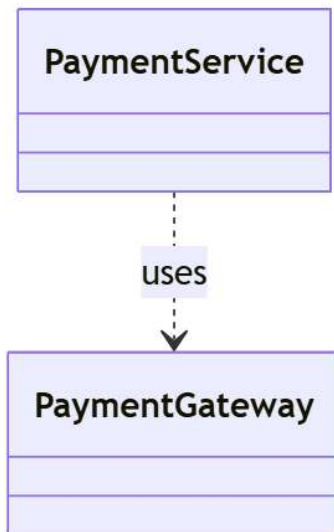
One class **temporarily uses** another class. No permanent reference to the other object is stored.

Example

A **PaymentService** uses a **PaymentGateway** to process a payment.

```
class PaymentService {
    void pay(PaymentGateway gateway) {
        gateway.process();
    }
}
```

Gateway is only used inside the method.



6. Interface Implementation (Realization)

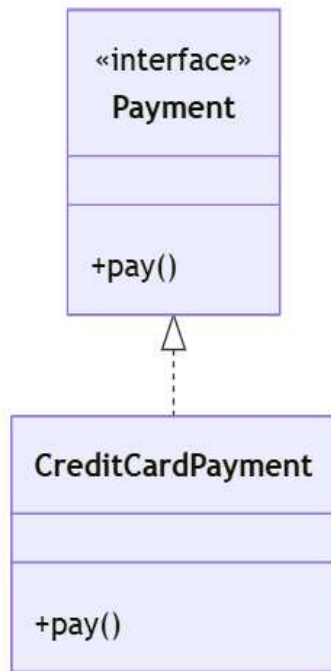
A class implements an interface.

Example

CreditCardPayment implements the Payment interface.

```
interface Payment {  
    void pay();  
}
```

```
class CreditCardPayment implements Payment {  
    @Override  
    public void pay() {  
    }  
}
```



Multiplicity

Multiplicity shows **how many objects participate in a relationship**.

Multiplicity Meaning

1	Exactly one
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many

Abstract Factory Pattern

The **Abstract Factory Pattern** is a creational design pattern that provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

Instead of creating individual objects directly, the client asks a factory to create an entire family of related products. The client works only with interfaces, while the concrete factory decides which objects to instantiate.

The main goal of this pattern is to ensure that related objects are created together and remain compatible with each other.

For example:

- A DominosFactory creates a Domino's Pizza, Garlic Bread, and Pepsi.
- A McDonaldsFactory creates a Burger, Fries, and Coke.

The client doesn't know or care about the concrete classes being created. It simply interacts with the factory and product interfaces.

Advantages of Abstract Factory Pattern

- **Loose Coupling**

- The client depends only on interfaces, not on concrete implementation classes.
- **Creates Consistent Product Families**
 - Ensures that related products belong to the same family and work together.
 - For example, a Domino's meal contains Domino's products only and doesn't accidentally mix items from McDonald's.
- **Easy to Switch Product Families**
 - Changing the entire family of products is as simple as changing the concrete factory.
 - For example, switching from DominosFactory to McDonaldsFactory changes the complete meal.
- **Follows Open/Closed Principle**
 - New product families can be added without modifying existing client code.
- **Centralizes Object Creation**
 - All object creation logic is kept inside factories instead of being scattered throughout the application.

When to Use

Use the Abstract Factory Pattern when:

- You need to create **multiple related objects** that are designed to work together.
- You want to hide object creation from the client.
- You want to work only with interfaces instead of concrete classes.

- You want to switch between different families of products at runtime or configuration time.
- You want to ensure that compatible products are always used together.
- Your application should remain independent of object creation logic.

Difference Between Factory Method and Abstract Factory

Factory Method

Creates one product.

Has one factory method.

Focuses on creating a single object.

Example: Pizza Factory creates Pizza.

Abstract Factory

Creates a family of related products.

Has multiple factory methods.

Focuses on creating compatible groups of objects.

Example: Domino's Factory creates Pizza, Garlic Bread, and Pepsi.

Real-Life Example: Amazon Marketplace

Imagine you're shopping on Amazon. Amazon sells products from different brands.

Suppose you choose the **Apple Store** on Amazon. Now every product you buy comes from Apple:

- MacBook
- iPhone

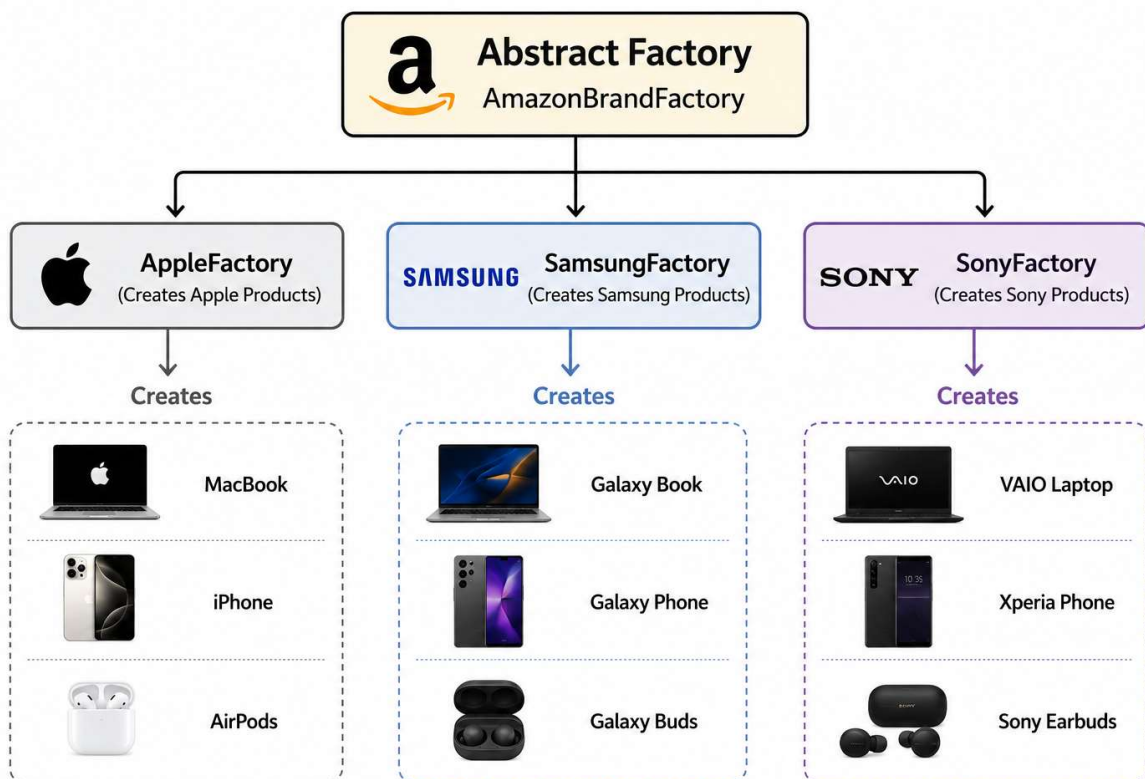
- AirPods

If you instead choose the **Samsung Store**, you get:

- Galaxy Book
- Galaxy Phone
- Galaxy Buds

Each brand provides a complete family of compatible products. As a customer, you don't know which concrete classes are created. You simply choose a brand and request the products you need. Changing the brand automatically changes the entire family of products.

This is exactly what the **Abstract Factory Pattern** does.



Step 1: Product Interfaces

Laptop

```
public interface Laptop {  
    String getModel();  
}
```

Phone

```
public interface Phone {  
    String getModel();  
}
```

Earbuds

```
public interface Earbuds {  
    String getModel();  
}
```

Even though Laptop, Phone, and Earbuds currently have the same method (**getModel()**), they represent different types of products.

We keep them separate because:

- They belong to different product families.
- The factory clearly knows what type of product it is creating.
- In the future, each interface can have its own methods (e.g., Laptop can have **getRAM()**, Phone can have **supports5G()**).

So, while the interfaces look similar today, they make the design cleaner, type-safe, and easier to extend later.

Step 2: Apple Products

```
public class MacBook implements Laptop {  
    @Override  
    public String getModel() {  
        return "MacBook Pro";  
    }  
}
```

```
public class iPhone implements Phone {  
    @Override  
    public String getModel() {  
        return "iPhone 17 pro";  
    }  
}
```

```
public class AirPods implements Earbuds {  
    @Override  
    public String getModel() {  
        return "AirPods Pro";  
    }  
}
```

Step 3: Samsung Products

```
public class GalaxyBook implements Laptop {  
    @Override  
    public String getModel() {  
        return "Galaxy Book 3 Pro";  
    }  
}
```

```
public class GalaxyPhone implements Phone {  
    @Override  
    public String getModel() {  
        return "Galaxy S23 Ultra";  
    }  
}
```

```
public class GalaxyBuds implements Earbuds {  
    @Override  
    public String getModel() {  
        return "Galaxy Buds Pro";  
    }  
}
```

Step 4: Abstract Factory

```
public interface BrandFactory {  
    Laptop createLaptop();  
    Phone createPhone();  
    Earbuds createEarbuds();  
}
```

```
}
```

Notice there is **no parameter**.

The factory knows what family to create.

Step 5: Apple Factory

```
public class AppleFactory implements BrandFactory {  
    @Override  
    public Laptop createLaptop() {  
        return new MacBook();  
    }  
  
    @Override  
    public Phone createPhone() {  
        return new iPhone();  
    }  
  
    @Override  
    public Earbuds createEarbuds() {  
        return new AirPods();  
    }  
}
```

Step 6: Samsung Factory

```
public class SamsungFactory implements BrandFactory {  
    @Override  
    public Laptop createLaptop() {  
        return new GalaxyBook();  
    }  
}
```



```

    }

    @Override
    public Phone createPhone() {
        return new GalaxyPhone();
    }

    @Override
    public Earbuds createEarbuds() {
        return new GalaxyBuds();
    }
}

```

Step 7: Client

```

public class Main {
    public static void main(String[] args) {
        shopFromBrand("Apple Store", new AppleFactory());
        shopFromBrand("Samsung Store", new SamsungFactory());
        shopFromBrand("Sony Store", new SonyFactory());
    }
}

```

```

    private static void shopFromBrand(String brandName,
BrandFactory factory) {
        System.out.println("----- Shopping from " + brandName +
" -----");

```

```

        Laptop laptop = factory.createLaptop();
        Phone phone = factory.createPhone();
        Earbuds earbuds = factory.createEarbuds();

```

```

        System.out.println("Laptop : " + laptop.getModel());
        System.out.println("Phone : " + phone.getModel());

```

```

        System.out.println("Earbuds : " + earbuds.getModel());
    }
}

```

```

/* *

```

Output :

```

----- Shopping from Apple Store -----
Laptop  : MacBook Pro
Phone   : iPhone 17 pro
Earbuds : AirPods Pro
----- Shopping from Samsung Store -----
Laptop  : Galaxy Book 3 pro
Phone   : Galaxy S23 Ultra
Earbuds : Galaxy Buds Pro
----- Shopping from Sony Store -----
Laptop  : Vaio Laptop
Phone   : Xperia 1 III
Earbuds : Sony Buds Pro

```

```

* */

```

Food Delivery Application

The Food Delivery App is an excellent Abstract Factory example because **each restaurant provides an entire family of related products (Main Course + Side + Drink)**. The client chooses a restaurant, and the factory ensures that all items belong to the same restaurant.

The app supports multiple restaurant brands:

Domino's

- Pizza
- Garlic Bread
- Pepsi

McDonald's

- Burger
- Fries
- Coke

KFC

- Chicken Bucket
- Chicken Popcorn
- Mountain Dew

Instead of creating each item manually, the client asks for a restaurant factory.

RestaurantFactory factory = **new** DominosFactory();

The factory creates an entire meal:

- Main Course → Pizza
- Side → Garlic Bread
- Drink → Pepsi

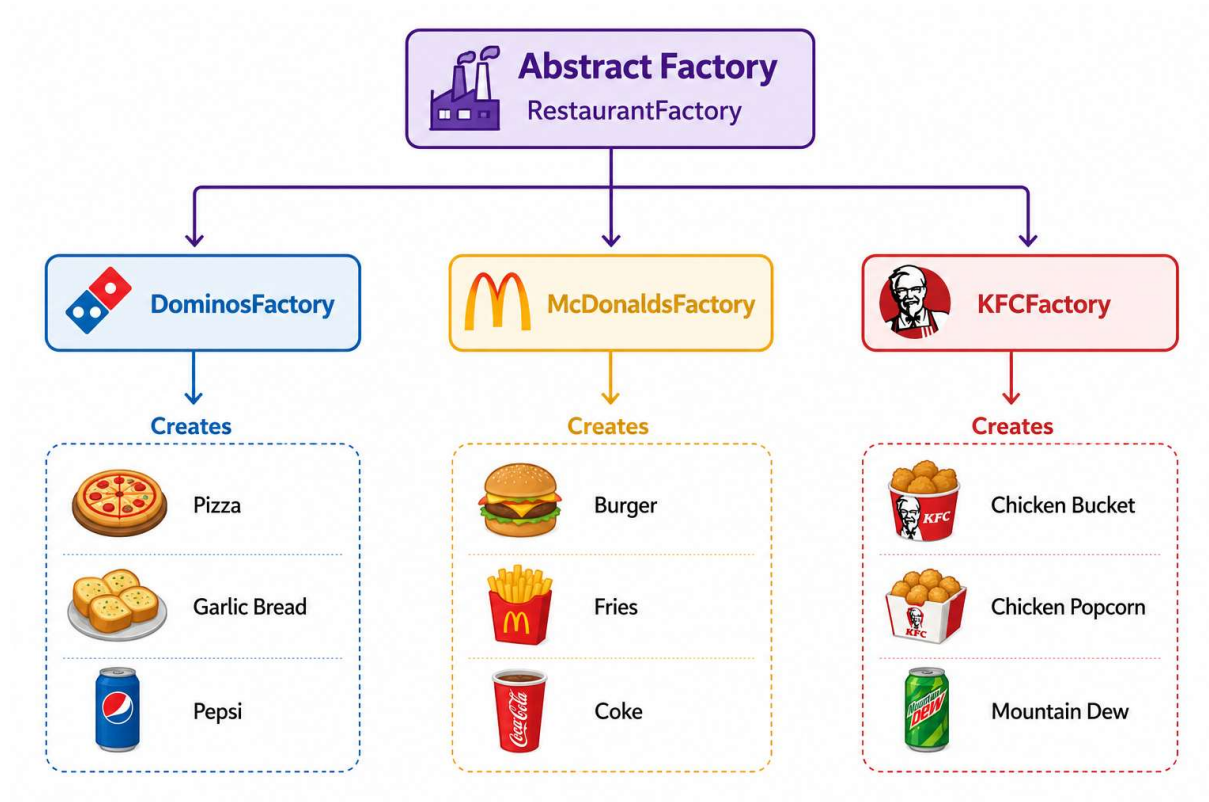
If the customer selects McDonald's, only the factory changes:

RestaurantFactory factory = **new** McDonaldsFactory();

Now the factory creates:

- Main Course → Burger
- Side → Fries
- Drink → Coke

The client code remains exactly the same. Only the factory changes, and the entire family of related products changes automatically.

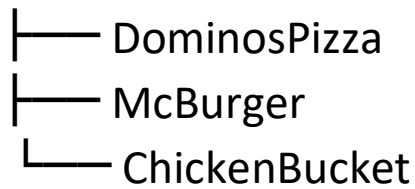


Project Structure

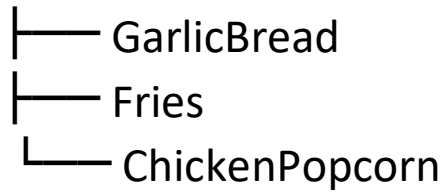
RestaurantFactory (Abstract Factory)



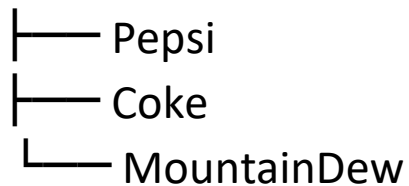
MainCourse (Product)



Side (Product)



Drink (Product)



Step 1. MainCourse Interface

```
public interface MainCourse {  
    String getName();  
    void addExtras();  
    int getCalories();  
}
```

Step 2. Side Interface

```
public interface Side {  
    String getName();  
    void cookingTemp();  
}
```

```
    boolean isSpicy();  
}
```

Step 3. Drink Interface

```
public interface Drink {  
    String getName();  
    void pour();  
    int getVolume();  
}
```

Step 4. Dominos Products

DominosPizza

```
public class DominosPizza implements MainCourse {  
    @Override  
    public String getName() {  
        return "Dominos Pizza";  
    }  
  
    @Override  
    public void addExtras() {  
        System.out.println("Adding cheese, pepperoni, and  
vegetables to Dominos Pizza.");  
    }  
  
    @Override  
    public int getCalories() {  
        return 300;  
    }  
}
```

GarlicBread

```
public class GarlicBread implements Side {  
    @Override  
    public String getName() {  
        return "Garlic Bread";  
    }  
  
    @Override  
    public void cookingTemp() {  
        System.out.println("Garlic Bread is baked at 180°C for 15  
minutes.");  
    }  
  
    @Override  
    public boolean isSpicy() {  
        return false;  
    }  
}
```

Pepsi

```
public class Pepsi implements Drink {  
    @Override  
    public String getName() {  
        return "Pepsi";  
    }  
  
    @Override  
    public void pour() {  
        System.out.println("Pouring Pepsi.");  
    }  
}
```

```
}

@Override
public int getVolume() {
    return 330; // volume in ml
}
}
```

Step 5. McDonald's Products

McBurger

```
public class McBurger implements MainCourse {
    @Override
    public String getName() {
        return "McBurger";
    }

    @Override
    public void addExtras() {
        System.out.println("Adding lettuce, tomato, and cheese to
McBurger.");
    }

    @Override
    public int getCalories() {
        return 450;
    }
}
```

Fries


```
public class Fries implements Side {  
    @Override  
    public String getName() {  
        return "Fries";  
    }  
  
    @Override  
    public void cookingTemp() {  
        System.out.println("Fries are cooked at 220 degrees  
Celsius.");  
    }  
  
    @Override  
    public boolean isSpicy() {  
        return false;  
    }  
}
```

Coke

```
public class Coke implements Drink {  
    @Override  
    public String getName() {  
        return "Coke";  
    }  
  
    @Override  
    public void pour() {  
        System.out.println("Pouring Coke");  
    }  
  
    @Override  
    public int getVolume() {  
        return 500; // volume in ml  
    }  
}
```

```
}
```

Step 6. KFC Products

ChickenBucket

```
public class ChickenBucket implements MainCourse {  
    @Override  
    public String getName() {  
        return "Chicken Bucket";  
    }  
  
    @Override  
    public void addExtras() {  
        System.out.println("Adding extra sauces to the chicken  
bucket.");  
    }  
  
    @Override  
    public int getCalories() {  
        return 1200; // Example calorie count for a chicken bucket  
    }  
}
```

ChickenPopcorn

```
public class ChickenPopcorn implements Side {  
    @Override  
    public String getName() {  
        return "Chicken Popcorn";  
    }  
}
```

```
@Override
public void cookingTemp() {
    System.out.println("Chicken Popcorn is cooked at 150
degrees Celsius.");
}
```

```
@Override
public boolean isSpicy() {
    return true;
}
}
```

MountainDew

```
public class MountainDew implements Drink {
    @Override
    public String getName() {
        return "Mountain Dew";
    }
}
```

```
@Override
public void pour() {
    System.out.println("Pouring Mountain Dew");
}
```

```
@Override
public int getVolume() {
    return 150; // volume in ml
}
}
```

Step 7. Abstract Factory

```
public interface RestaurantFactory {  
    MainCourse createMainCourse();  
    Side createSide();  
    Drink createDrink();  
}
```

Step 8. Concrete Factories

DominosFactory

```
public class DominosFactory implements RestaurantFactory {  
    @Override  
    public MainCourse createMainCourse() {  
        return new DominosPizza();  
    }  
  
    @Override  
    public Side createSide() {  
        return new GarlicBread();  
    }  
  
    @Override  
    public Drink createDrink() {  
        return new Pepsi();  
    }  
}
```

McDonaldsFactory

```
public class McDonaldsFactory implements RestaurantFactory {  
    @Override
```

```
public MainCourse createMainCourse() {  
    return new McBurger();  
}  
  
@Override  
public Side createSide() {  
    return new Fries();  
}  
  
@Override  
public Drink createDrink() {  
    return new Coke();  
}  
}
```

KFCFactory

```
public class KFCFactory implements RestaurantFactory {  
    @Override  
    public MainCourse createMainCourse() {  
        return new ChickenBucket();  
    }  
  
    @Override  
    public Side createSide() {  
        return new ChickenPopcorn();  
    }  
  
    @Override  
    public Drink createDrink() {  
        return new MountainDew();  
    }  
}
```

Step 9. Client

```
public class Main {
    public static void main(String[] args) {
        orderMeal("Dominos", new DominosFactory());
        orderMeal("McDonald's", new McDonaldsFactory());
        orderMeal("KFC", new KFCFactory());
    }

    private static void orderMeal(String restaurantName,
    RestaurantFactory factory) {
        System.out.println("\n----- " + restaurantName +
        " -----");

        MainCourse mainCourse = factory.createMainCourse();
        Side side = factory.createSide();
        Drink drink = factory.createDrink();

        // Main Course
        System.out.println("Main Course : " +
        mainCourse.getName());
        mainCourse.addExtras();
        System.out.println("Calories : " +
        mainCourse.getCalories());
        System.out.println();

        // Side
        System.out.println("Side : " + side.getName());
        side.cookingTemp();
        System.out.println("Spicy : " + side.isSpicy());
        System.out.println();

        // Drink
        System.out.println("Drink : " + drink.getName());
        drink.pour();
    }
}
```

```

        System.out.println("Volume      : " + drink.getVolume() + "
ml");
    }
}

```

```

/* *

```

Output :

----- Dominos -----
Main Course : Dominos Pizza
Adding cheese, pepperoni, and vegetables to Dominos
Pizza.

Calories : 300

Side : Garlic Bread
Garlic Bread is baked at 180°C for 15 minutes.
Spicy : false

Drink : Pepsi
Pouring Pepsi.
Volume : 330 ml

----- McDonald's -----
Main Course : McBurger
Adding lettuce, tomato, and cheese to McBurger.
Calories : 450

Side : Fries
Fries are cooked at 220 degrees Celsius.
Spicy : false

Drink : Coke
Pouring Coke
Volume : 500 ml

----- KFC -----

Main Course : Chicken Bucket
Adding extra sauces to the chicken bucket.
Calories : 1200

Side : Chicken Popcorn
Chicken Popcorn is cooked at 150 degrees Celsius.
Spicy : true

Drink : Mountain Dew
Pouring Mountain Dew
Volume : 150 ml

** */*